Assignment 3

Reinforcement Learning Programming - CSCN8020 By:

Jarius Bedward

Link to Repo: https://github.com/JariusBedward-8841640/DQN_Assignment3_JB.git

March 27, 2026

Note:

Contents of this PDF is the same as jupyter notebook + README file in repo

CSCN8020 Assignment 3
Project Members:
1. Jarius Bedward #8841640

Project Summary:
This project focuses on implementing a Deep Q-Network(DQN) agent to play the Pong environment from OpenAI Gym.
The project is different from traditional Q-Learning with discrete state spaces, pong presents a high-dimensional visual input in the form of image frames. To address this, a Convolutional Neural Netwrok(CNN) is used to approximate the Q-function, which lets the agent learn directly.
The objective is to train the agent to learn an optimal policy to control the paddle and successfully return the ball, maximizing cumulative reward over time.
The project also tests the impact of key hyperparametrs like mini-batch size and target network update frequency to see how they affect things like learning stability, convergence, and overall performance as well as seeing what might be the best combination

Requirements:
- Python 3.11
- pip install -r requirements.txt
- AutoROM --accept-license

🎯 How to Run:

1. Clone this repo (git clone <repo-url> cd <repo-folder)
2. Install Required Dependencies: "pip install -r requirements.txt"
3. Run AutoROM --accept-license
4. Open the jupyter notebook
5. Run all the cells within the notebook

Workflow/Methodology:

1. **Environment Setup**
- The PongDeterministic-v4 environment is initialized from OpenAI Gym
- The agent interacts with the environment through a discrete action space
2. **Preprocessing**
- Raw game frames are preprocessed by cropping, converting to grayscale, and resizing
- Four consecutive frames are stactked to capture temporal information such as motion and velocity
3. **Model Architecture**
- A CNN is used to process visual input and approximate Q-values for each action
4. **Training Process**
- The agent interacts with the environment using an ε-greedy policy
- Experiences are stored in a replay buffer
- Mini-batches are sampled to train the netwrok using the DQN update rule
5. **Target Network**
- A separate target network is used to stabilize training
- The target network is periodically updated with thew eights of the amin netwrok
6. **Experimentation**
- Controlled experiments are conducted by modifying the batch size and target network update frequency

  - Each experiment is ran independently to isolate the effect of each parameter
7. **Evaluation**
  - Performance is evaluated using total reward per episode and average reward over the last 5 episodes
  - Results are visualized and compared to determine the best configuration

Reinforcement Learning Formulation
- **Agent**: The DQN agent controlling the right paddle
- **Environment**: PongDeterministic-v4
- **State Space**: Preprocessed image frames (84 x 80 x 4 stack)
- **Action Space**: Discrete actions (NOOP, FIRE, LEFT,RIGHT, etc)
- **Policy**: ε-greedy policy
- **Value Function**: Q(s, a) approximated using a neural network
- **Learning Type**: Model-free, off-policy, Value-based Reinforcement Learning

The goal of teh agent is to learn a policy that maximizes cumulative reward over time by selecting the optimal actions based on the current visual state

The DQN update rule is based on the Bellman Equation:
$Q(s,a) \leftarrow Q(s,a) + \alpha [r + \gamma \max_{a'} Q(s',a') - Q(s,a)]$

Where:
- r is the reward received
- y is the discount factor (0.95)
- max Q(s',a') represents the estimated future reward

Unlike traditional Q-learning the Q-function is approximated using a CNN, allowing the agent to handle high dimensional visual inputs

Experience replay and a target network are used to improve training stability and help reduce correlations between consecutive updates
Log File Interpretation

Final Network Architecture

A Deep Q-Network (DQN) was used and is a Convolutional Neural Network (CNN) designed to process visual input from the Pong environment and approximate the Q-function

Input Representation
The input to the network consists of four consecutive preprocessed frames, stacked together to capture motion information. Each frame has a resolution of 84 x80, resulting in an input shape of:

(84 x 80, 4)

Convolutional Layers
The network uses three layers to extract spatial features from the inpt:

1. **Conv Layer 1**
  - 32 filters
  - Kernel size: 8 × 8
  - Stride: 4
  - Activation: ReLU

2. **Conv Layer 2**
  - 64 filters
  - Kernel size: 4 × 4
  - Stride: 2
  - Activation: ReLU

3. **Conv Layer 3**
  - 64 filters
  - Kernel size: 3 × 3
  - Stride: 1
  - Activation: ReLU

These layers slowly reduce the spatial dimensions while learning important visual features such as paddle position and ball trajectory

Fully Connected layers

The output layers of the convolutional layers is flattened which is to say we convert the multi dimensional feature map into a one dimensional vector and passed through two fully connected layers:

- Dense layer with 512 units and ReLU activation
- Output layer with size equal to the number of possible actions

The final output represents the Q-values for each possible action, which shows the expected future reward of taking each action in the given state.

Output Layer

The network outputs a vector of Q-values:

$Q(s, a_1), Q(s, a_2), ..., Q(s, a_n)$

where each value corresponds to one of the available actions in the Pong environment

The agent selects actions using an ε-greedy policy, choosing the action with the highest Q-value during exploitation

Metrics, observations & comments of parameter change (W/ plots)

- Across all three experiments, the agent initially performs very poorly, with average rewards close to -21, this shows near-random gameplay. Over time, all the different configurations show some sort of improvement which confirms that the agent is learning meaningful policies.

- Experiment 1 demonstrates gradual improvement, ending around an average reward of approximately -16. While learning occurs, the progress is slow and unstable, suggesting inefficient parameter settings.

- Experiment 2 shows the best improvement out of all 3, it reached an average reward near -11. This configuration shows faster convergence and the most stable performance, indicating that the chosen batch size and target update frequency effectively balance learning stability and adaptability.

- Experiment 3 also improves over time but converges to a lower performance level -14. The learning was less consistent, with higher variance in rewards. If more episodes were involved in training experiment 3 could have overtaken experiment 2 in being the best in terms of average rewards.

Simulation + Log Interpretation
- The simulation/training ran for 300 episodes for each experiment.
I made the training go for 300 episodes because after testing at 100 episodes and lower no significant changes were seen and so
I found that by choosing 300 some actual changes began to occur.

- The log file shows that for experiment 1 it had a slow improvement and was quite noisy. It also took long to stablize.
Experiment 2 had a strong upward trend and consistently improved.
Experiment 3 was just as strong but was less stable than experiment 2.

Best combo of batch size and update rate for target network

- Based on the factors evaluated which are; speed, stability, and final performance, the best configuration is:
Experiment 2(Batch Size =16, Target Update = 10)

- The combination achieved:
  - The highest average reward (-11)
  - Faster learning compared to the other configurations
  - More stable convergence with less variance

  The improved performance may suggest that:
  - A moderate batch size provides stable gradient updates
  - A balanced target update frequency prevents instability while allowing learning to adapt

Final Conclusion
The results demonstrate that hyperparameter does significantly impact Deep Q-Network performance. In this case the batch size and target network update frequency determines both learning speed and stability. The optimal configuration achieves a balance between frequent learning updates and stable target estimation.

Extra Notes:

🤝 Contributing
- This is an Assignment developed for CSCN8020. If any questions arise do not hesitate to contact the project member.